

pifanctl v1

Release Acceptance Field Manual

Repeatable hardware, failure, rollback, and fleet-load checks for the 1.0.0 release decision.

DECISION: NOT READY FOR STABLE RELEASE. Hardware-specific acceptance gates remain open. This document defines the procedure and evidence record; it does not claim that unmeasured tests passed.

Release snapshot

Field	Value
Target / current version	1.0.0 / 1.0.0-alpha.1
Current source main	887f6f1
Trial namespace and image	pifanctl-v1-trial / ghcr.io/jyje/pifanctl-issue:887f6f1-py312
Runtime chart source	887f6f1; successful Python 3.12 build and image checks
Cluster	MicroK8s, four ARM64 nodes, Kubernetes v1.36.2

Read-only live baseline - 2026-10-04

Observation	Observed value
Nodes	raspi-40, raspi-41, raspi-50, raspi-51
Topology	One shared rack fan cools four named Nodes
Actuator	r4spi-rack-fan on raspi-40, RPi.GPIO BCM 18, 1000 Hz
Status	Argo Synced/Healthy; operator, four agents, worker, Fan, and Zone Ready
Command / observation	24.375% requested duty at 49.05 C, observed 2026-10-04 12:18:16 UTC
Physical evidence	Post-upgrade visual confirmation pending; no RPM or waveform record

Current main-source candidate is deployed to the isolated trial. Previous trial image 6a0f5a5-py312 remains the immediate rollback target. This snapshot is a baseline only, not a release acceptance result.

1. Evidence and electrical safety

Evidence rules

- Record candidate commit, image digest, chart version, topology hash, Node identity, and timestamp for every run.
- Use an isolated test rack or maintenance window for worker stops, network partitions, reboots, or power removal.
- Keep a person at the hardware for fan and power tests. Record direct fan rotation; use a tachometer and oscilloscope or logic analyzer when available.
- Use Pass only when criteria and evidence are recorded. Otherwise record Fail, Blocked, or Not run.
- Requested PWM duty is not measured RPM. Stop on unexpected heat, cooling loss, unstable power, or unexplained signal state.

Electrical fail-open requirements

100% is a requested PWM duty, not a measured voltage, fan RPM, or guarantee that the fan has power. Signal state during process termination, board reboot, disconnected wiring, or controller power loss depends on the Pi, driver, fan input polarity, and external circuit. A floating input or GPIO HIGH must not be assumed to mean maximum cooling.

Before using pifanctl as a cooling safety control, measure signal and rotation on each supported hardware combination and provide an independent, correctly rated fan supply. The fan input needs a verified hardware default that requests the fan safe maximum when software no longer controls the signal. The project prescribes no universal pull-up, pull-down, or polarity. Software cannot correct fan-supply loss or mechanical failure.

Hardware inventory

Field	Record before test
Pi model/revision; OS/kernel/firmware	_____
Node name and UID	_____
PWM driver; exact GPIO or chip/channel	_____
Fan model; rated voltage/current	_____
Fan supply; fuse/current limit	_____
PWM input type, polarity, frequency range	_____
Ground/reference; signal wiring; hardware default	_____
Tachometer or rotation measurement	_____

2. PWM hardware checks

A. Raspberry Pi 4 GPIO PWM

1. Record Pi model/revision, OS/kernel, fan model, power, wiring, pin, polarity, and instrument setup.
2. With safe fan power and an observer present, start the worker on the actuator Node and capture process start through driver initialization.
3. Verify initial full duty is applied before telemetry is accepted. Measure frequency, duty, polarity, voltage levels, startup latency, and physical rotation.
4. Apply known settings. Confirm measured duty tracks each request, and the fan starts reliably from rest at configured start duty without stalling.
5. Stop normally. Verify configured exit duty and intended safe state. Repeat at least three cold starts and three normal stops.

Pass criteria: waveform and physical behavior match the fan specification and configuration at every tested point. Startup and shutdown behavior is repeatable. A 100% software request alone is not a pass.

B. Raspberry Pi 5 kernel sysfs PWM

1. Verify actual kernel overlay, PWM chip/channel, pin mux, polarity, and frequency. The sample chip 0/channel 2 is illustrative, not a board guarantee.
2. Record channel ownership and verify there is no competing writer.
3. Repeat waveform, start duty, stable duty points, normal shutdown, cold starts, and rotation measurements from procedure A.
4. Reboot the Pi and capture signal from power-on until the worker initializes. Verify all intermediate defaults.

Pass criteria: chip/channel mapping is verified, one writer owns the channel, and waveform, physical behavior, and boot/shutdown defaults are repeatable and safe.

Do not infer Pi model from a Kubernetes Node name. Record the physical board revision and the exact fan and driver.

3. Failure behavior

C. Process exit, crash, and watchdog

Test graceful stop and forced termination separately. Normal shutdown can run cleanup; SIGKILL cannot. Record signal and fan behavior for both. In the isolated trial only, interrupt fresh plan/heartbeat delivery and confirm the 120-second watchdog without affecting unrelated workloads.

Pass: graceful stop applies documented exit state; forced termination and stale heartbeat reach the independently designed safe electrical default; operator reports stale/unready; recovery requires fresh matching plan and heartbeat. Behavior that depends on Python cleanup is not protection against SIGKILL.

D. Node reboot and power loss

On an isolated actuator host, test orderly reboot, abrupt controller power loss, and fan-supply interruption as separate runs. Observe the fan and capture signal voltage through each transition. Verify independent fan supply behavior. Restore one failure at a time and confirm the worker reclaims the hardware lock without competing writers.

Pass: every failure has a measured and documented outcome. If cooling cannot continue during a failure, state that limit and keep the stable-release gate open unless product safety requirements explicitly accept it.

E. Network partition and telemetry loss

In the isolated trial, interrupt worker-to-operator heartbeat and Prometheus telemetry separately. Record fault start, failsafe time, requested and measured PWM, /readyz, metrics, and recovery. Keep expected membership unchanged.

Pass: missing/stale telemetry and expired heartbeat do not lower duty; worker remains unready/degraded with a visible reason; recovery uses fresh data and matching configuration identity.

Run field	Record
Candidate commit / image digest	_____
Failure and start/end time	_____
Requested / measured duty; fan observation	_____
Recovery evidence and artifact checksum	_____

4. Rollback and scale

F. Migration and rollback

1. Capture the archived v0 configuration and checksum. Inventory every process that can write each PWM channel.
2. Compare a read-only v1 topology plan with the physical rack. Keep the fan at full speed during handover.
3. Stop the legacy writer and verify it released the hardware before starting v1. Confirm the host-wide lock rejects a second writer.
4. Exercise rollback: stop v1, verify driver close and lock release, restore archived v0, and verify original controller/cooling behavior.
5. Repeat for each supported driver/topology, including one Pi fan and shared rack fans.

Pass: no overlapping writers; handover and rollback are repeatable; intended configuration returns on all Nodes; archive is usable.

G. Fleet-load acceptance

Declare the supported fleet size: Nodes, CoolingZones, Fans, worker endpoints, and scrape interval. Measure operator CPU/memory, API requests, reconcile duration, status writes, Prometheus latency, scrape gaps, and heartbeat age at that size during steady state and recovery. Use representative synthetic telemetry first; do not generate production API load without a maintenance plan.

Pass: all workers meet freshness limits; stale state is never Ready; API and Prometheus load fit the documented cluster budget; status writes do not create unbounded etcd churn.

Evidence record

Run	Candidate / image	Procedure	Result	Evidence hash / reviewer
1	_____	_____	Not run	_____
2	_____	_____	Not run	_____
3	_____	_____	Not run	_____

Release gate summary

Gate	State	Evidence needed
Pi 4 GPIO and Pi 5 sysfs	Open	Measured waveform and physical behavior on candidate
Process, node, power, network	Open	Signal/rotation, status, and recovery records
Migration and rollback	Open	Repeatable results for supported configurations
Fleet load	Open	Measurements at a declared fleet size
Safety limits documented	Documented; hardware validation open	Measured defaults, independent supply, explicit failure limits

Release decision

Do not label 1.0.0 stable until each applicable gate is backed by evidence for the exact release candidate. List untested hardware/topologies as unsupported. Preserve the alpha rollback archive until the decision is recorded.

References: docs/v1/runtime.md | docs/v1/README.md | PLAN.md | RPi.GPIO project | Linux kernel PWM interface